

Autonomiczny samochód wyścigowy z użyciem Proximal Policy Optimisation

Projekt – Głębokie Uczenie ze Wzmocnieniem

1 Opis problemu

Celem projektu jest nauczenie agenta autonomicznego pokonywania okrążeń na dwuwymiarowym torze wyścigowym bez żadnej zakodowanej na stałe strategii jazdy. Środowisko opiera się na symulacji Pygame. Agent otrzymuje wyłącznie informację wizualną – wycinek obrazu wyśrodkowany na pojeździe – i uczy się sterowania wyłącznie na podstawie sygnału nagrody.

2 Środowisko

2.1 Symulacja

Tor jest stałą jednopętlową trasą o rozdzielczości 810×810 pikseli. Silnik fizyczny pochodzi z klasy `AbstractCar`: pojazd ma ciągłą prędkość i kąt obrotu oraz pięć dyskretnych akcji (*przód*, *tył*, *lewo*, *prawo*, *stop*). Zderzenie z granicą toru wywołuje `bounce()`, co neguje prędkość.

Aby zmaksymalizować przepustowość treningu, zegar symulacji jest ustawiony na (FPS = 10000).

2.2 Cykl epizodu

Epizod zaczyna się, gdy samochód zostaje umieszczony w pozycji startu o współrzędnych (180, 200). Kończy się – a pojazd natychmiast wraca na start – w dwóch przypadkach:

- **Ukończenie okrążenia:** samochód przejeżdża przez linię mety z właściwego kierunku. Zdarzenie to jest odnotowywane jako *sukces*; licznik okrążeń rośnie, a trening jest kontynuowany bez przerwy.
- **Przekroczenie limitu czasu:** pojazd przez n kolejnych kroków nie zalicza żadnego nowego punktu kontrolnego. Zdarzenie

jest odnotowywane jako *porażka*. Wartość zmiennej n była zmienna zależnie od etapu treningu, początkowo wynosiła ona 128 potem zwiększoną tą wartość do 1024.

Bufor ramek oraz licznik postępu na checkpointach są czyszczone przy każdym resecie gry, tak aby nowy epizod startował z czystego stanu.

3 Przestrzeń obserwacji

3.1 Mapa toru

Zamiast pikseli RGB środowisko utrzymuje statyczną mapę `TRACK_MAP` o kształcie (H, W) (ta sama rozdzielczość co okno gry). Jest ona obliczana raz przy starcie na podstawie kanału alfa sprite'a krawężnika:

$$\text{TRACK_MAP}[r, c] = \begin{cases} 0,0 & \text{jeśli } \alpha[r, c] > 0 \text{ (ściana)} \\ 0,5 & \text{w przeciwnym razie (jezdnia)} \end{cases} \quad (1)$$

Przy generowaniu każdej ramki do mapy dołączane są dynamicznie cztery wartości semantyczne:

Wartość	Znaczenie
0,0	Ściana / poza obrębem toru
0,5	Jezdnia
0,75	Kolejne 30 checkpointów
1,0	Karoseria pojazdu

3.2 Wycinek egocentryczny

Z mapy semantycznej wycinany jest obszar 256×256 pikseli za pomocą odwrotnej transformacji afinicznej (`cv2.warpAffine` z flagami `WARP_INVERSE_MAP` | `INTER_NEAREST`), tak że:

- Kierunek jazdy samochodu zawsze wskazuje *górną* obrazu wyjściowego.
- Pojazd jest zakotwiczony blisko *dolnego środka* ramki (wiersz $r_{\text{car}} = 256 - h/2 - 2$, gdzie h to wysokość sprite'a), co maksymalizuje widoczność toru przed autem.

Macierz transformacji ma postać:

$$M = \begin{bmatrix} \cos \theta & \sin \theta & t_x \\ -\sin \theta & \cos \theta & t_y \end{bmatrix} \quad (2)$$

$$t_x = c_x - (\cos \theta \cdot 128 + \sin \theta \cdot r_{\text{car}}) \quad (3)$$

$$t_y = c_y - (-\sin \theta \cdot 128 + \cos \theta \cdot r_{\text{car}}) \quad (4)$$

gdzie (c_x, c_y) to środek samochodu, a θ jego kąt obrotu.

Aby nanieść punkt kontrolny ze współrzędnych światowych (w_x, w_y) na obraz wyjściowy, transformacja jest odwracana analitycznie (obróć jest ortogonalny, więc odwrotność to transpozycja):

$$\begin{pmatrix} \text{col} \\ \text{row} \end{pmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{pmatrix} w_x - t_x \\ w_y - t_y \end{pmatrix} \quad (5)$$

Każdy z kolejnych 30 punktów kontrolnych jest rysowany jako wypełnione koło o promieniu 5 pikseli i wartości 0,75.

3.3 Stos klatek

Cztery kolejne semantyczne klatki są łączone wzdłuż osi kanałów, dając tensor stanu $s \in \mathbb{R}^{4 \times 256 \times 256}$. Jeśli dostępnych jest mniej niż cztery klatki (np. na początku epizodu), pierwsza dostępna klatka jest powielana.

4 Przestrzeń akcji

Agent wybiera jedną z pięciu dyskretnych akcji w każdym kroku:

Indeks	Nazwa	Efekt
0	przód	Przyspieszenie
1	tył	Hamowanie / cofanie
2	lewo	Skręt w lewo
3	prawo	Skręt w prawo
4	stop	Stopniowe spowalnianie

5 Funkcja nagrody

Skalarna nagroda r_t w każdym kroku składa się z trzech składników.

Nagroda za postęp. Iloczyn skalarny wektora kierunku jazdy $\hat{h}$ i jednostkowego wektora $\hat{d}$ wskazującego na kolejny punkt kontrolny, skalowany aktualną prędkością v :

$$r_{\text{postęp}} = (\hat{h} \cdot \hat{d}) \cdot \frac{v}{8} \quad (6)$$

gdzie $\hat{h} = (-\sin \alpha, -\cos \alpha)$ (konwencja Pygame, oś y skierowana w dół), a $\hat{d}$ to znormalizowany wektor od środka samochodu do najbliższego checkpointu.

Kara za bliskość ściany. Trzy promienie (przodem, przód-lewo, przód-prawo) mierzą odległość do krawężnika. Jeśli którakolwiek jest poniżej 15 pikseli, naliczana jest ujemna kara:

$$r_{\text{ściana}} = \sum_{d \in \{d_f, d_{fl}, d_{fr}\}} \mathbf{1}[d < 15] \cdot \frac{d - 15}{25} \quad (7)$$

Uderzenie w ścianę jest odnotowywane do statystyk, jeśli $r_{\text{ściana}} < 0$.

Kara za czas. Stała wartość $-0,025$ na krok zniechęca do stania w miejscu:

$$r_t = r_{\text{postęp}} + r_{\text{ściana}} - 0,025 \quad (8)$$

6 Architektura sieci

Polityka i funkcja wartości dzielą konwolucyjny kręgosłup (*backbone*) i rozgałęziają się na oddzielne głowice MLP. Utrzymywane są trzy niezależne optymalizatory Adam (po jednym na podsieci), ale wszystkie wykonują krok na tej samej zagregowanej stracie.

6.1 Backbone

Warstwa	Parametry
Conv2d → GroupNorm → GELU	4→32, $k=5$, $s=4$
Conv2d → GroupNorm → GELU	32→64, $k=5$, $s=2$
Conv2d → GroupNorm → GELU	64→96, $k=5$, $s=1$
Conv2d → GroupNorm → GELU	96→96, $k=3$, $s=2$
Flatten	—

GroupNorm(1, C) traktuje cały map przestrzenny każdego kanału jako jedną grupę, co odpowiada LayerNorm po wymiarach (C, H, W) .

6.2 Głowica aktora

$$\begin{aligned} &\xrightarrow{\text{LazyLinear}(128)} \text{GELU} \\ &\xrightarrow{\text{Linear}(64)} \text{GELU} \\ &\xrightarrow{\text{Linear}(5)} \text{Softmax} \rightarrow \pi(a|s) \end{aligned}$$

6.3 Głowica krytyka

$$\begin{aligned} &\xrightarrow{\text{LazyLinear}(128)} \text{GELU} \\ &\xrightarrow{\text{Linear}(64)} \text{GELU} \\ &\xrightarrow{\text{Linear}(1)} V(s) \end{aligned}$$

(feature_dim = 256, więc rozmiary warstw ukrytych wynoszą kolejno 128 i 64.)

7 Algorytm uczenia: PPO

7.1 Zbieranie danych (rollout)

Agent wchodzi w interakcję ze środowiskiem przez $N = 2048$ kroków przed jakąkolwiek aktualizacją parametrów. W każdym kroku zapisywana jest krotka $(s_t, a_t, \log \pi_{\text{old}}(a_t|s_t), r_t, \text{done}_t)$ na CPU.

7.2 Uogólniona estymacja przewagi (GAE)

Wartości $V(s_t)$ dla wszystkich stanów rollout są obliczane w jednym wsadowym przebiegu do przodu (bez gradientów). Wartość bootstrapowa $V(s_N)$ jest szacowana, jeśli ostatni krok nie był terminalny. Przewagi są obliczane metodą GAE [?]:

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t) \quad (9)$$

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \gamma = 0,99, \quad \lambda = 0,95 \quad (10)$$

Zwroty to $\hat{R}_t = \hat{A}_t + V(s_t)$. Przewagi są następnie normalizowane do zerowej średniej i jednostkowej wariancji w obrębie całego rollout.

7.3 Aktualizacja PPO

Przez $K = 8$ epok rollout jest tasowany i przetwarzany w mini-batchach po 32 próbki. Łączna strata to:

$$\mathcal{L} = \mathcal{L}_{\text{aktor}} + \mathcal{L}_{\text{krytyk}} + \mathcal{L}_{\text{entropia}} \quad (11)$$

Strata aktora (przycięty surogat).

$$\mathcal{L}_{\text{aktor}} = -\mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right] \quad (12)$$

gdzie $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$ oraz $\varepsilon = 0,1$.

Strata krytyka. Błąd średniokwadratowy między przewidywanymi wartościami a zwrotami GAE:

$$\mathcal{L}_{\text{krytyk}} = \mathbb{E} \left[(V_{\theta}(s_t) - \hat{R}_t)^2 \right] \quad (13)$$

Bonus entropii.

$$\mathcal{L}_{\text{entropia}} = -\beta_H \mathbb{E} [\mathcal{H}(\pi_{\theta}(\cdot|s_t))], \quad (14)$$

Gdzie współczynnik β był dostosowywany zależnie od etapu treningu ręcznie (początkowo był wysoki wraz z czasem był zmniejszany).

7.4 Optymalizacja

Trzy niezależne optymalizatory Adam ze wspólnym współczynnikiem uczenia obsługują odpowiednio: backbone, głowicę aktora i głowicę krytyka. Normy gradientów są przycinane do wartości 3,0 niezależnie dla każdej podsieci. Wartość stałej uczenia była zmienna zależnie od etapu treningu.

7.5 Zestawienie hiperparametrów

Hiperparametr	Wartość
Długość rollout N	2048
Liczba epok PPO K	8
Rozmiar mini-batcha	32
γ	0,99
λ	0,95
Parametr przycinania ε	0,1
Przycinanie gradientów	3,0
Wymiar cech	256
Limit bez postępu	2048 kroków